University Mohammed VI Polytechnic

College of Computing

Java Programming & Software Development

Prof. Fahd Kalloubi
Fahd.kalloubi@um6p.ma

- Associate professor in Machine Learning and Big Data
- Ex Adjunct professor at euro-Mediterranean university of Fez
- Affiliate professor at UM6P – College of Computing
- Ex Professor at ENSIAS
- Research interests: Machine learning, Graph Neural Networks, NLP, Deep Learning, Big Data

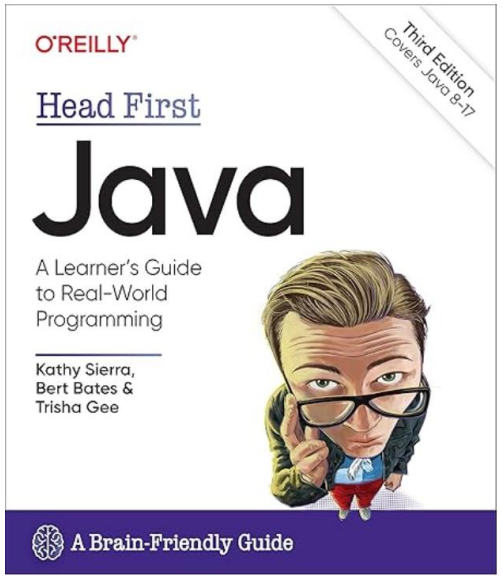


Lecture Topics

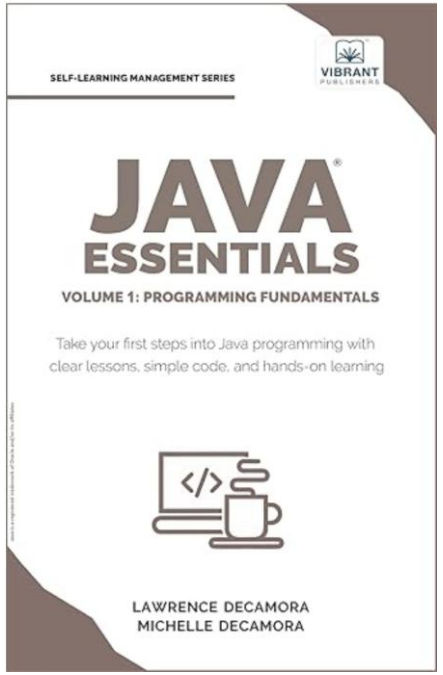
1. Introduction to List and ArrayList
2. Arrays vs. ArrayLists
3. Arrays and ArrayLists of reference types
4. Big O notation of ArrayLists operations
5. LinkedList
6. ArrayList vs. LinkedList
7. Iterator vs. ListIterator

Literature

Head First Java: A Brain-Friendly Guide, 3rd Edition
Kathy Sierra, Bert Bates, Trisha Gee
O'REILLY
June 21, 2022



Java Essentials, Volume 1: Programming Fundamentals: Learn Java, Variables, Control Flow, and Methods with Hands-On Coding Exercises and Examples for Beginners
Dr. Lawrence G. Decamora III, Michelle C. Decamora
Vibrant Publishers
September 25, 2025



This Lecture



Instructional Objectives

After completing this part, you should be able to:

- Differentiate between Arrays and ArrayLists
- Illustrate the use different operations using ArrayLists
- Understand ArrayList capacity and Constant amortized time using Big O notation
- Illustrate the use of LinkedList
- Differentiate between ArrayList and LinkedList
- Illustrate the use of Iterator and ListIterator



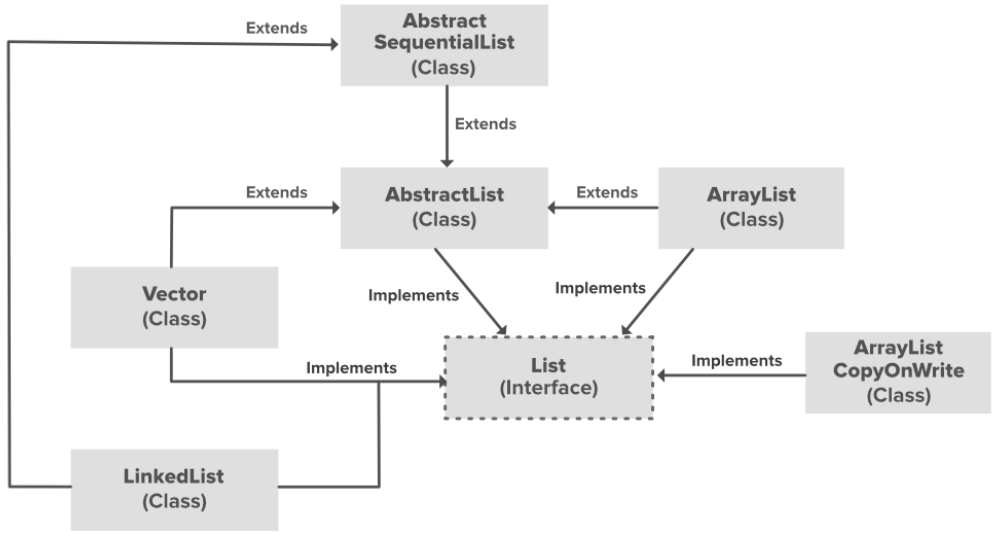
Learning outcomes

At the end part, you are expected to:

- Implement java programs using ArrayList and LinkedList collections
- Understand when to use ArrayList and LinkedList
- Implement traversal over records using Iterator and ListIterator
- Implement LIFO and FIFO data collection using java collections

List and ArrayList

- A List is a special type in Java
- For now, we can say a List Interface describes a set of method signatures that all List classes are expected to have.
  - See: List methods in Java's API documentation page
- The ArrayList is a class that maintains an array in memory that's actually bigger than what we need, in most cases.
  - It keeps track of the capacity or maximum size of the array in memory.
  - But it also keeps track of the elements that've been assigned or set, which is the size of the ArrayList.
- As elements are added to an ArrayList, its capacity may need to grow. This occurs for you automatically, behind the scenes.
  - This is why the ArrayList is resizeable.



Arrays vs ArrayLists

Feature	Array	ArrayList
primitives' types supported	Yes	**No**
indexed	Yes	Yes
ordered by index	Yes	Yes
duplicates allowed	Yes	Yes
nulls allowed	Yes, for non-primitive types	Yes
resizable	**No**	Yes
mutable	Yes	Yes
inherits from java.util.Object	Yes	Yes
implements List interface	No	**Yes**

Instantiating without Values

Instantiating Arrays	Instantiating ArrayLists
`String[] array = new String[10];`	`ArrayList<String> arrayList = new ArrayList<>();`
An array of 10 elements is created, all with null references. The compiler will only permit Strings to be assigned to the elements.	An empty ArrayList is created. The compiler will check that only Strings are added to the ArrayList.

- An array requires square brackets in the declaration.
- On the right-hand side of the equals sign, square brackets are also required with a size specified inside.
- An ArrayList should be declared with the type of element for the ArrayList in angle brackets.

Instantiating with Values

Instantiating Arrays

```
String[] array = new String[] {"first", "second", "third"};
```

An array of 3 elements is created, with

```
elements[0] = "first"

elements[1] = "second"

elements[2] = "third
```

Alternately, we can use this array initializer (anonymous array).

```
String[] array = {"first", "second", "third"};
```

Instantiating Lists and Array Lists

```
ArrayList<String> arrayList = new ArrayList<>(List.of("first",
        "second", "third"));
```

An ArrayList can be instantiated by passing another list to it as we show here.

We can use the List.of() factory method, which uses variable arguments, to create a pass through immutable list.

- You can use an array initializer to populate array elements during array creation.
- This feature lets you pass all the values in the array as a comma delimited list in curly braces.
- When you use an array initializer in a declaration statement, you can use what's called the anonymous version, as I show here.
- You can use an ArrayList constructor that takes a collection or a list of values during ArrayList creation.
  - The List.of method can be used to create such a list, with a variable argument list of elements.

Element information

	Accessing Array Element data	Accessing ArrayList Element data
	Example Array: `String[] arrays = {"first", "second", "third"};`	Example ArrayList: `ArrayList<String> arrayList = new ArrayList<>(List.of("first", "second", "third"));`
Index value of first element	`0`	`0`
Index value of last element	`arrays.length - 1`	`arrayList.size() - 1`
Retrieving number of elements:	`int elementCount = arrays.length;`	`int elementCount = arrayList.size();`
Setting (assigning an element)	`arrays[0] = "one";`	`arrayList.set(0, "one");`
Getting an element	`String element = arrays[0];`	`String element = arrayList.get(0);`

- The number of elements is fixed when an array is created.
- You can get the size of the array from the attribute length on the array instance.
- Array elements are accessed with the use of square brackets and an index that ranges from 0 to one less than the number of elements.
- The number of elements in an ArrayList may vary and can be retrieved with a method on the instance, named size().
- ArrayList elements are accessed with get and set methods, also using an index ranging from 0 to one less than the number of elements.

String representation for Single Dimension Arrays and ArrayLists

Array

Array Creation Code

```
String[] arrays = {"first", "second", "third"};
```

Printing Array Elements

```
System.out.println(Arrays.toString(arrays));
```

ArrayList

ArrayList Creation Code

```
ArrayList<String> arrayList = new ArrayList<>(List.of("first",
        "second", "third"));
```

Printing ArrayList elements

```
System.out.println(arrayList);
```

- ArrayLists come with built-in support for printing out elements, including nested lists.
- Arrays don't though, so you need to call Arrays.toString, passing the array as an argument.
- This slide shows examples of single dimension arrays and ArrayLists.

String representation for Multi-Dimensional Arrays and ArrayLists

Array

Array Creation Code

```
String[][] array2d = {
        {"first", "second", "third"},
        {"fourth", "fifth"}
};
```

Printing Array Elements

```
System.out.println(Arrays.deepToString(array2d));
```

ArrayList

ArrayList Creation Code

```
ArrayList<ArrayList<String>> multiDList = new ArrayList<>();
```

Printing ArrayList elements

```
System.out.println(multiDList);
```

- Here, I show examples of multi-dimensional arrays and ArrayLists, and how to print the elements in each.
- A multi-dimensional ArrayList simply has a type, which in itself is an ArrayList.
- For a multi-dimensional array, you need to call the Arrays.deepToString method, passing the array as an argument.
- For nested ArrayLists, we can still just pass the ArrayList instance directly, to System.out.println

Finding an element in an Array or ArrayList

Arrays methods for finding elements	ArrayList methods for finding elements

`int binarySearch(array, element)`
** Array MUST BE SORTED
Not guaranteed to return index of first element if there are duplicates
	`boolean contains(element)`
	`boolean containsAll(list of elements)`
	`int indexOf(element)`
	`int lastIndexOf(element)`

- For arrays, you can use the binarySearch method to find a matching element, although this method requires that the array be sorted first.
- In addition, if the array contains duplicate elements, the index returned from this search is not guaranteed to be the position of the first element.

Finding an element in an Array or ArrayList

Arrays methods for finding elements	ArrayList methods for finding elements

`int binarySearch(array, element)`
** Array MUST BE SORTED
Not guaranteed to return index of first element if there are duplicates
	`boolean contains(element)`
	`boolean containsAll(list of elements)`
	`int indexOf(element)`
	`int lastIndexOf(element)`

- For the ArrayList, we have several methods.
  - You can use contains or containsAll, which simply returns a boolean if a match or matches were found.
  - In addition, like the String and StringBuilder, we have the methods, indexOf and lastIndexOf, which will return the index of the first or last match.
  - When a -1 is returned from these methods, no matching entry was found.

Sorting

Array	ArrayList

`String[] arrays = {"first", "second", "third"};`
 	
`Arrays.sort(arrays);`
	`ArrayList<String> arrayList = new ArrayList<>(List.of("first",`
	`"second", "third"));`
	
	`arrayList.sort(Comparator.naturalOrder());`
	`arrayList.sort(Comparator.reverseOrder());`

You can only sort arrays of elements that implement Comparable. We'll be discussing this in a future section. Character Sequence classes, like String and StringBuilder meet this requirement.	You can use the sort method with static factory methods to get Comparators.

- We can use the Arrays.sort method, for arrays with numeric primitive types and wrapper classes, as well as Strings and StringBuilders.
- For the ArrayList, we can use the sort method again for numeric wrapper classes, Strings and StringBuilders.
- You pass a Comparator type argument to ArrayList's sort method that specifies how the sort should be performed.
- You call static methods on the Comparator type to get a Comparator for either a natural order, or reverse order sort.

Array as an ArrayList

```
String[] originalArray = new String[] {"First", "Second", "Third"};
var originalList = Arrays.asList(originalArray);
```

- There are times when you'll want to switch between an Array and an ArrayList, and there is support for this on both the Arrays class and the ArrayList class.
- The Arrays.asList method returns an ArrayList backed by an array.
- You can think of this conceptually, as putting an ArrayList wrapper of sorts around an existing array.
- Any change made to the List is a change to the array that backs it.
- This also means that an ArrayList created by this method is not resizable.

Creating Special Kinds of Lists

Using Arrays.asList	Using List.of
Returned List is NOT resizeable, but is mutable.	**Returned List is IMMUTABLE.**
`var newList = Arrays.asList("Sunday", "Monday", "Tuesday");`	`var listOne = List.of("Sunday", "Monday", "Tuesday");`
`String[] days = new String[] {"Sunday", "Monday", "Tuesday"};` `List<String> newList = Arrays.asList(days);`	`String[] days = new String[] {"Sunday", "Monday", "Tuesday"};` `List<String> listOne = List.of(days);`

- This slide demonstrates two ways to create a list. From elements or from an array of elements.
- Both are static factory methods on types.
- The first is the asList method on the Arrays class, and it returns a special instance of a List that is not resizable, but is mutable.
- The second is the of method on the List interface, and it returns a special instance of a List that is immutable.
- Both support variable arguments, so you can pass a set of arguments of one type, or you can pass an array.

Creating Arrays from ArrayLists

```
ArrayList<String> stringLists = new ArrayList<>(List.of("Jan","Feb","Mar"));
String[] stringArray = stringLists.toArray(new String[0]);
```

- To create an array from an ArrayList we use the method toArray.
- This method takes one argument which should be an instance of a typed array.
- This method returns an array of that same type.
- If the length of the array you pass has more elements than the list, extra elements will be filled with the default values for that type.
- If the length of the array you pass has less elements than the list, the method will still return an array, with the same number of elements in it, as the list.
- In the example shown here, I pass a String array with zero as the size, but the array returned has three elements, which is the number of elements in the list.

The ArrayList Challenge

The challenge is to create an interactive console program, with a menu of options as shown here, and functionality that I will describe now:

```
Available actions:
0 - to shutdown
1 - to add item(s) to list (comma delimited list)
2 - to remove any items (comma delimited list)
Enter a number for which action you want to do:
```

Using the Scanner class, solicit a menu choice, 0, 1, or 2, and process the item accordingly.

The ArrayList Challenge

- The list will be a grocery list and should be an ArrayList.
- You should use methods on the ArrayList to add items, remove items, check if an item is already in the list, and print a sorted list.
- You should print the list, sorted alphabetically, after each operation.
- You shouldn't allow duplicate items in the list.

Array of primitive values

- When an array of primitive types is allocated, space is allocated for all of it's elements contiguously, as shown here.
- You can see from this slide that we have an array of seven integers.
- The index position is in the left column, and that's the number we use to access a specific array value.
- So the first element, when we use index position 0, this will retrieve the value 34.
- The addresses I show here are memory addresses represented by these numbers.

Index	Value	Address
0	34	100
1	18	104
2	91	108
3	57	112
4	453	116
5	68	120
6	6	124

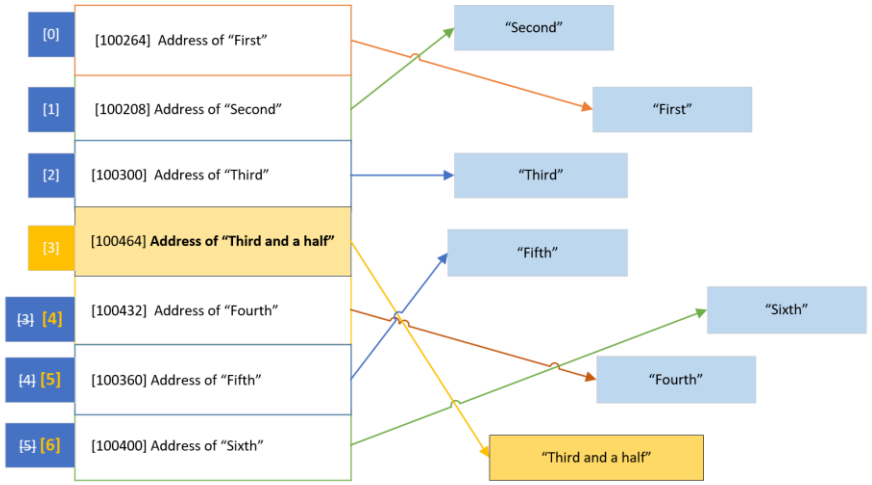
Array of primitive values

- If 100 is the address of an integer and we know an integer is 4 bytes, then the address of the next integer if it's contiguous would be 104, as I show here for the second element.
- Java can use simple math using the index and the address of the initial element in the array to get the address and retrieve the value of the element.

Index	Value	Address
0	34	100
1	18	104
2	91	108
3	57	112
4	453	116
5	68	120
6	6	124

Arrays and ArrayLists of reference types

- For reference types (meaning anything that's not a primitive type) like a String or any other object, the array elements aren't the values, but the addresses of the referenced object or String.
- This means our objects aren't stored contiguously in memory, but their addresses are, in the array behind the ArrayList.
- And again, the addresses can be easily retrieved with a bit of math, if we know the index of the element.
  - But to remove an element, the referenced addresses have to be re-indexed or shifted to remove an empty space.
  - And when adding an element, the array that backs the ArrayList might be too small and might need to be reallocated
  - the String, "Third and a half', represents a new element I want to insert at index position 3, meaning that all the elements below this point need to be moved and re-indexed.



ArrayList capacity

- An ArrayList is created with an initial capacity depending on how many elements we create the list with, or if you specify a capacity when creating the list
- In this example, I show an ArrayList that has a capacity of 10 because I'm passing 10 in the constructor of this list. I then add 7 elements.

```
ArrayList<Integer> intList = new ArrayList<>(10);
for (int i = 0; i < 7; i++) {
    intList.add((i + 1) * 5);
}
```

Index	0	1	2	3	4	5	6	7	8	9
Value	5	10	15	20	25	30	35			

ArrayList capacity

- I can add 3 more elements using the ArrayList add method, and the array that is used to store the data doesn't need to change.

```
ArrayList<Integer> intList = new ArrayList<>(10);
for (int i = 0; i < 7; i++) {
    intList.add((i + 1) * 5);
}

intList.add(40);
intList.add(45);
intList.add(50);
```

- The elements at indices 7, 8, and 9 get populated.

Index	0	1	2	3	4	5	6	7	8	9
Value	5	10	15	20	25	30	35	40	45	50

ArrayList capacity is reached

- But if the number of elements exceeds the current capacity, Java needs to reallocate memory to fit all the elements, and this can be a costly operation, especially if your ArrayList contains a lot of items.

```
ArrayList<Integer> intList = new ArrayList<>(10);
for (int i = 0; i < 7; i++) {
    intList.add((i + 1) * 5);
}
intList.add(40);
intList.add(45);
intList.add(50);

intList.add(55);            // This add exceeds the ArrayList capacity,
                            // assuming an initial capacity of 10,
                            // as an example.
```

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Value	5	10	15	20	25	30	35	40	45	50	55				

ArrayList capacity is reached

- So now, if our code simply calls add on this ArrayList, the next operation is going to create a new array, with more elements, but needs to copy the existing 10 elements over.
- Then the new element is added. You can imagine this add operation costs more, in both time and memory, than the previous add methods did.
- When Java reallocates new memory for the ArrayList, it automatically sets the capacity to a greater capacity.
- From their own documentation, Java states that, "The details of the growth policy are not specified beyond the fact that adding an element, has constant amortized time cost".
  - Let's start with how to determine cost, which in this case is generally considered in terms of time, but may include memory usage and processing costs, etc.

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Value	5	10	15	20	25	30	35	40	45	50	55				

Big O Notation

- Big O approximates the cost of an operation for a certain number of elements called n.
- Cost is usually determined by the time it takes, but it can include memory usage and complexity, for example.
- As n (the number of elements) gets bigger, an operation's cost can stay the same.
- But cost often grows as the number of elements grow.
  - Costs can grow linearly, meaning the cost stays in step with the magnitude of the number of elements.
  - Or costs can grow exponentially or by some other non-linear fashion.
- In a perfect world, an operation's time and complexity would never change. This ideal world, in Big O Notation is O(1), sometimes called constant time.
- In many situations, an operation's cost is in direct correlation to the number of elements, n. In Big O Notation this is O(n), sometimes called linear time.

Big O Notation

- So, if we have 10 elements, the cost is 10 times what it would be for 1 element because the operation may have to execute some functions up to 10 times vs. just once, and 100 times for 100 elements, for example.
- O(n) is generally our worst case scenario for List operations, but there are Big O Notations, for worse performers.

Constant Amortized Time Cost

- Another scenario is the one the Java docs declared for the growth of the ArrayList that adding an element has constant amortized time cost.
- In our case, we'll designate this constant amortized time as O(1)*.
  - This means that in the majority of cases, the cost is close to O(1), but at certain intervals, the cost is O(n).
- If we add an element to an ArrayList where the capacity of the List is already allocated and space is available, the cost is the same each time, regardless of how many elements we add.
- But as soon as we reach the capacity and all the elements (all n elements) need to be copied in memory, this single add would have a maximum cost of O(n).
- After this operation, that forced a reallocation, any additional add operations go back to O(1), until the capacity is reached again.
- As the expensive intervals decrease, the cost gets closer to O(1), so we give it the notation O(1)*.

ArrayList Operations – Big O

- In general, the cost will be something in between for the contains method because the element will be found somewhere between the first and nth (or last) element.
- You'll notice that the indexed methods are usually O(1), remembering that finding an element by its index is a simple calculation.
- It only gets costly with indexed add or remove methods, if the ArrayList needs to be re-indexed, or re-sized.

Operation	Worst Case	Best Case
add(E element)	O(1)*	
add(int index, E element)	O(n)	O(1)*
contains(E element)	O(n)	O(1)
get(int index)	O(1)	
indexOf(E Element)	O(n)	O(1)
remove(int index)	O(n)	O(1)
remove(E element)	O(n)	
set(int index, E element)	O(1)	

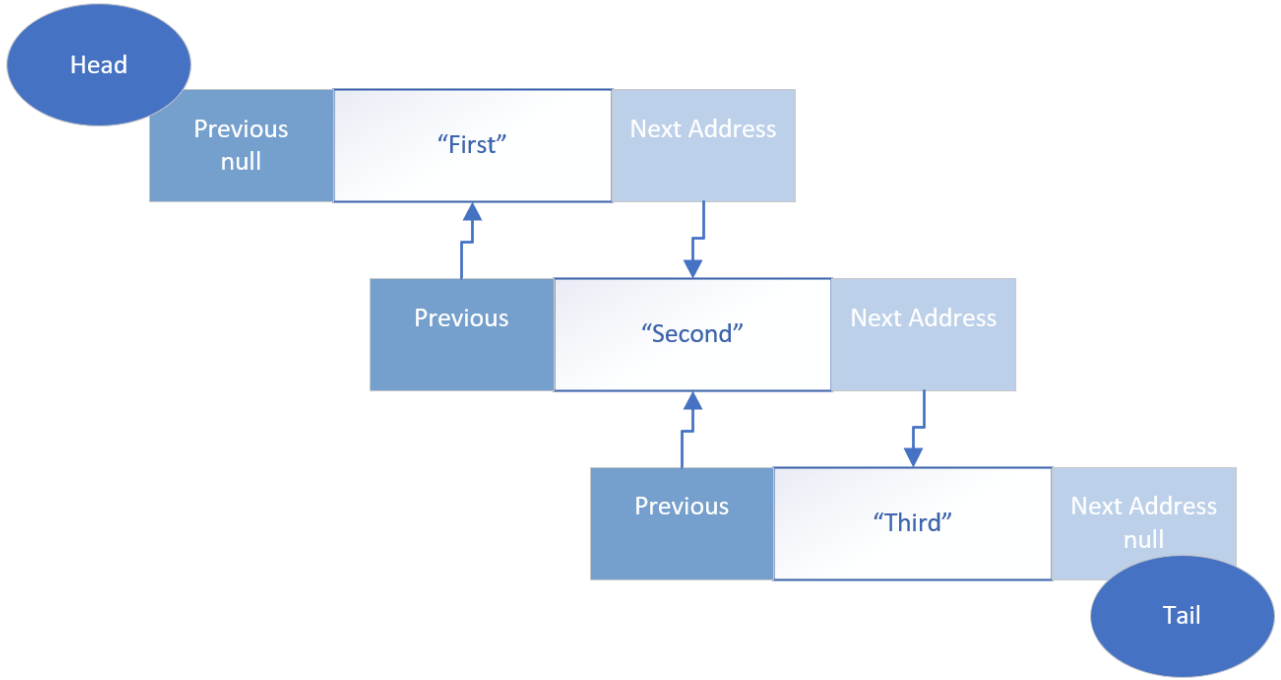
O(1) - constant time - operation's cost (time) should be constant regardless of number of elements.

O(n) - linear time - operation's cost (time) will increase linearly with the number of elements n.

O(1)* - constant amortized time - somewhere between O(1) and O(n), but closer to O(1) as efficiencies are gained.

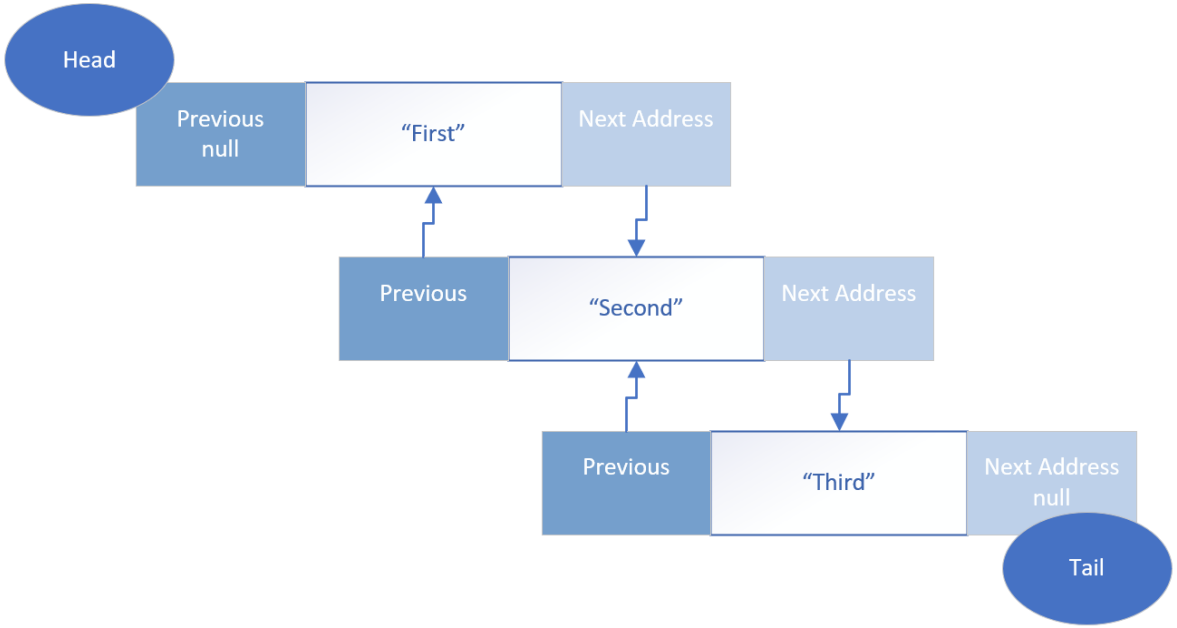
LinkedList

- The LinkedList is not indexed at all.
- There is no array storing the addresses in a neat, ordered way, as we saw with the ArrayList.
- Instead, each element that's added to a linked list forms a chain and the chain has links to the previous element, and the next element.



LinkedList

- This architecture is called a doubly linked list, meaning an element is linked to the next element, but it's also linked to a previous element, in this chain of elements.
- The beginning of the chain is called the head of the list, and the end is called the tail.
- This can also be considered a queue, in this case, a double ended queue, because we can traverse both backwards and forwards, through these elements.



LinkedList – retrieval cost

- Getting an element from the list or setting a value of element, isn't just simple math anymore with the LinkedList type.
- To find an element, you'd need to start at the head or tail, and check if the element matches or keep track of the number of elements traversed, if we are matching by an index because the index isn't stored as part of the list.
- For example, even if you know you want to find the 5th element, you'd still have to traverse the chain this way to get that fifth element.
- This type of retrieval is considered expensive in computer currency, which is processing time and memory usage.
- On the other hand, inserting and removing an element, is much simpler for this type of collection.

LinkedList – inserting and removing cost

- In contrast to an ArrayList, inserting or removing an item in a LinkedList is just a matter of breaking two links in the chain, and re-establishing two different links.
- No new array needs to be created, and elements don't need to be shifted into different positions.
- A reallocation of memory to accommodate all existing elements is never required.
- For a LinkedList, inserting and removing elements is generally considered cheap in computer currency, compared to doing these functions in an ArrayList.

ArrayList and LinkedList Operations – Big O

- This table shows the Big O values for the most common shared List operations or methods for both types.
- For a LinkedList, adding elements to the start or end of the List will almost always be more efficient than an ArrayList.

	Linked List		ArrayList	
Operation	**Worst Case**	**Best Case**	**Worst Case**	**Best Case**
add()	O(1)		O(1)*	
add(int index, E element)	O(n)	O(1)	O(n)	O(1)*
contains(E element)	O(n)	O(1)	O(n)	O(1)
get(int index)	O(n)	O(1)	O(1)	
indexOf(E Element)	O(n)	O(1)	O(n)	O(1)
remove(int index)	O(n)	O(1)	O(n)	O(1)
remove(E element)	O(n)	O(1)	O(n)	
set(int index, E element)	O(n)	O(1)	O(1)	

O(1) - constant time - operation's cost (time) should be constant regardless of number of elements.

O(n) - linear time - operation's cost (time) will increase linearly with the number of elements n.

O(1)* - constant amortized time - somewhere between O(1) and O(n), but closer to O(1) as efficiencies are gained.

ArrayList and LinkedList Operations – Big O

- When removing elements, a LinkedList will be more efficient because it doesn't require re-indexing, but the element still needs to be found using the traversal mechanism, which is why it is O(n), as the worst case.
- Removing elements from the start or end of the List will be more efficient for a LinkedList.

	Linked List		ArrayList	
Operation	**Worst Case**	**Best Case**	**Worst Case**	**Best Case**
add()	O(1)		O(1)*	
add(int index, E element)	O(n)	O(1)	O(n)	O(1)*
contains(E element)	O(n)	O(1)	O(n)	O(1)
get(int index)	O(n)	O(1)	O(1)	
indexOf(E Element)	O(n)	O(1)	O(n)	O(1)
remove(int index)	O(n)	O(1)	O(n)	O(1)
remove(E element)	O(n)	O(1)	O(n)	
set(int index, E element)	O(n)	O(1)	O(1)	

O(1) - constant time - operation's cost (time) should be constant regardless of number of elements.

O(n) - linear time - operation's cost (time) will increase linearly with the number of elements n.

O(1)* - constant amortized time - somewhere between O(1) and O(n), but closer to O(1) as efficiencies are gained.

Wrap up – ArrayList vs LinkedList

- The ArrayList is usually the better default choice for a List, especially if the List is used predominantly for storing and reading data.
- If you know the maximum number of possible items, then it's probably better to use an ArrayList, but set it's capacity.
- This code demonstrates how to set the capacity of your ArrayList to 500,000.

```
int capacity = 500_000;
ArrayList<String> stringArray = new ArrayList<>(capacity);
```

- An ArrayList's index is an int type, so an ArrayList's capacity is limited to the maximum number of elements an int can hold, Integer.MAX_VALUE = 2,147,483,647.

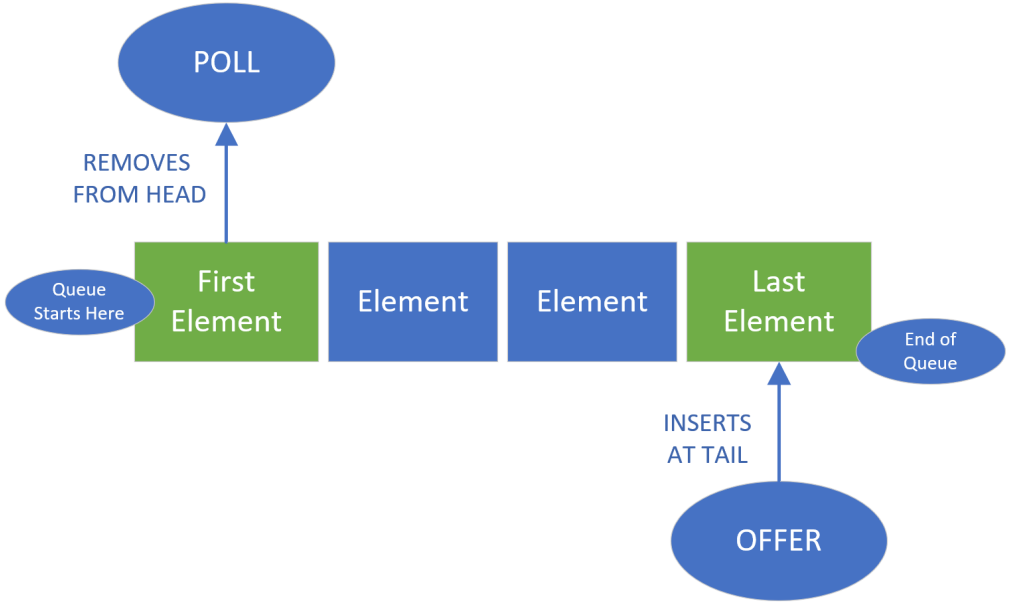
Wrap up – ArrayList vs LinkedList

- You may want to consider using a LinkedList if you're adding and processing or manipulating a large amount of elements, and the maximum elements isn't known but may be great, or if your number of elements may exceed Integer.MAX_VALUE.
- A LinkedList can be more efficient when items are being processed predominantly from either the head or tail of the list.

Feature	**ArrayList**	**LinkedList**
Internal structure	Dynamic array	Doubly linked list
Random access	Fast (O(1))	Slow (O(n))
Insert/delete in middle	Slow	Faster
Memory overhead	Low	High
Iteration speed	Fast	Slower
Best use case	Frequent access	Frequent insertion/deletion

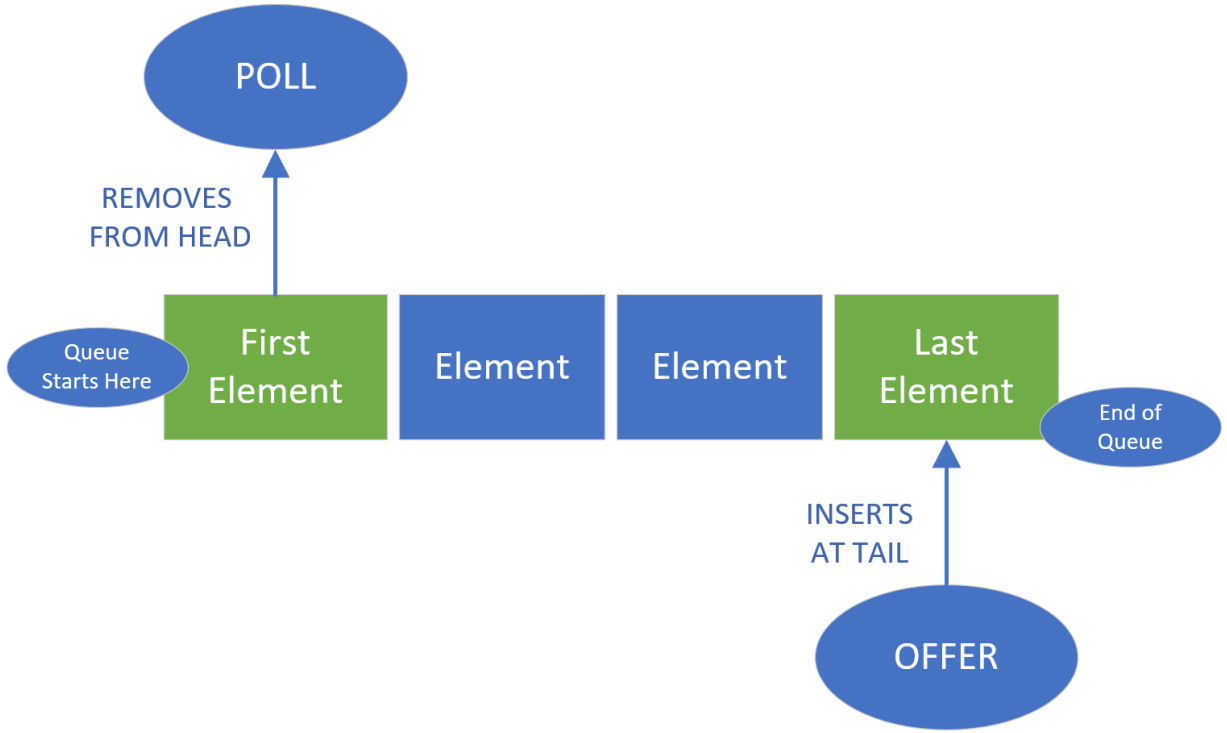
A Queue is a First-In, First-Out (FIFO) Data Collection

- An ArrayList is implemented on top of an array, but a LinkedList is a doubly linked list.
- Both implement all of List's methods, but the LinkedList also implements the Queue and Stack methods (Deque Interface) as well.
- When you think of a queue, you might think of standing in line.
  - When you get in a line or a queue, you expect that you'll be processed, in relationship to the first person in line.
  - We call this a First-in First-out. or FIFO data collection.



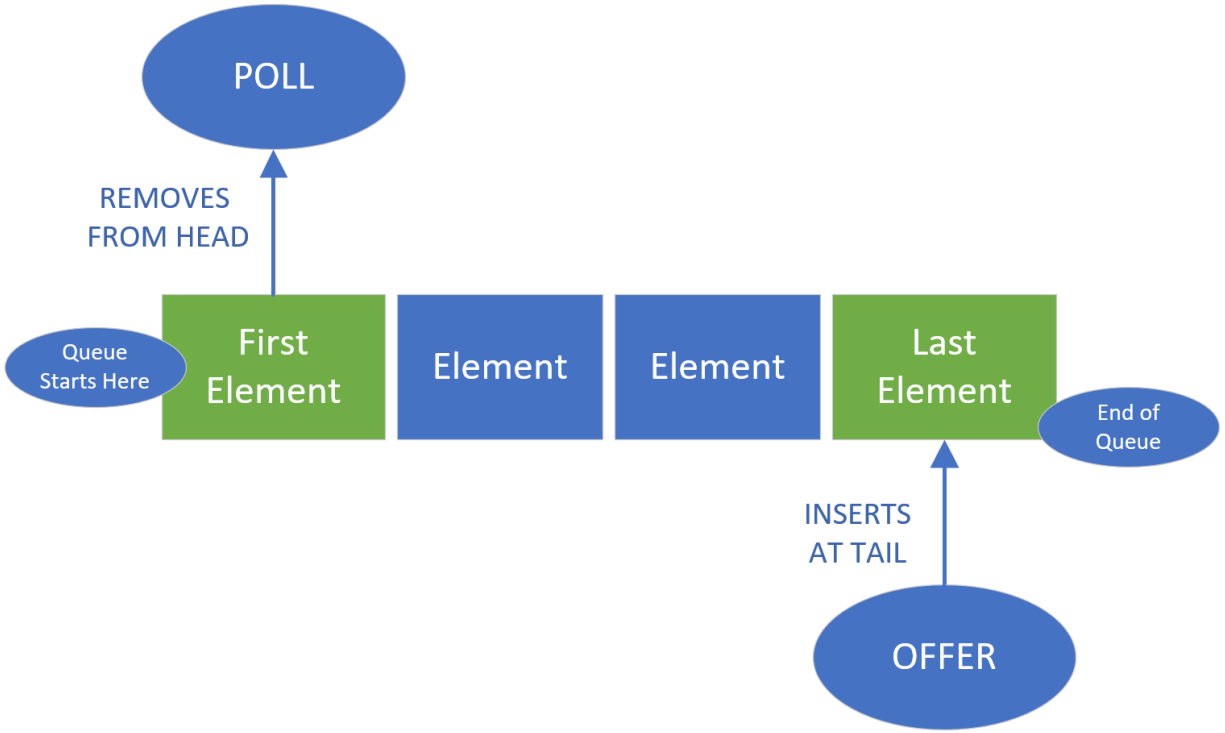
A Queue is a First-In, First-Out (FIFO) Data Collection

- If you want to remove an item, you poll the queue, getting the first element or person in the line.
- If you want to add an item, you offer it onto the queue, sending it to the back of the line.
- Single-ended queues always process elements from the start of the queue.



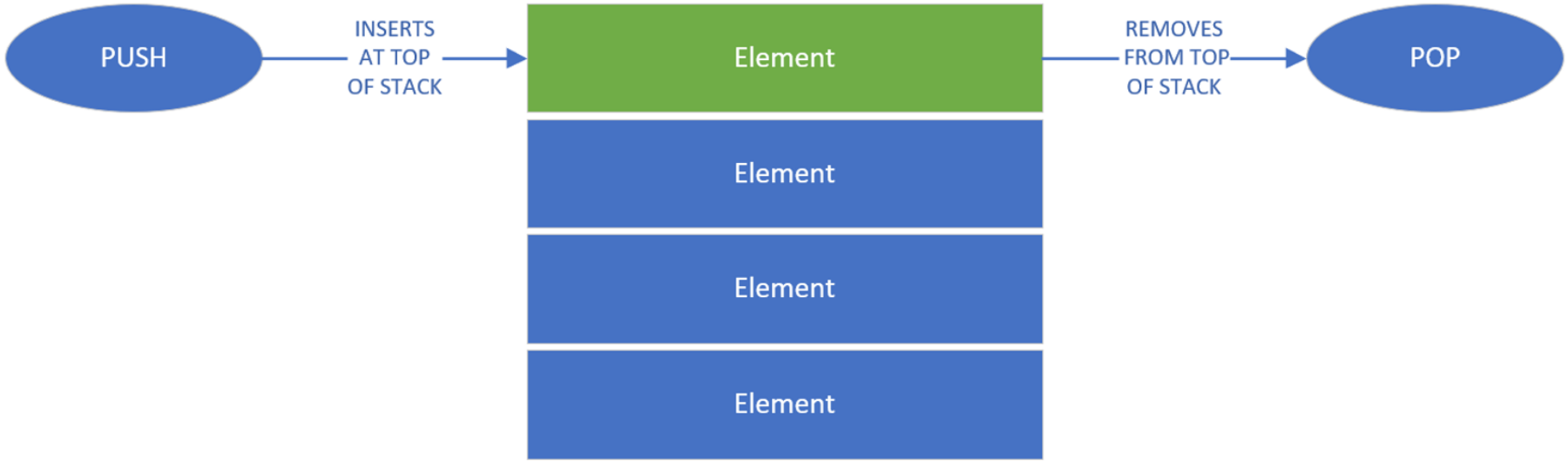
A Queue is a First-In, First-Out (FIFO) Data Collection

- A double-ended queue allows access to both the start and end of the queue.
- A LinkedList can be used as a double ended queue.



A Stack is a Last-In, First-Out (LIFO) Data Collection

- When you think of a stack, you can think of a vertical pile of elements, one on top of another
- When you add an item, you push it onto the stack.
- If you want to get an item, you'll take the top item, or pop it from the stack.
- We call this a Last-In First-out, or LIFO data collection.
- A LinkedList can be used as a stack as well.

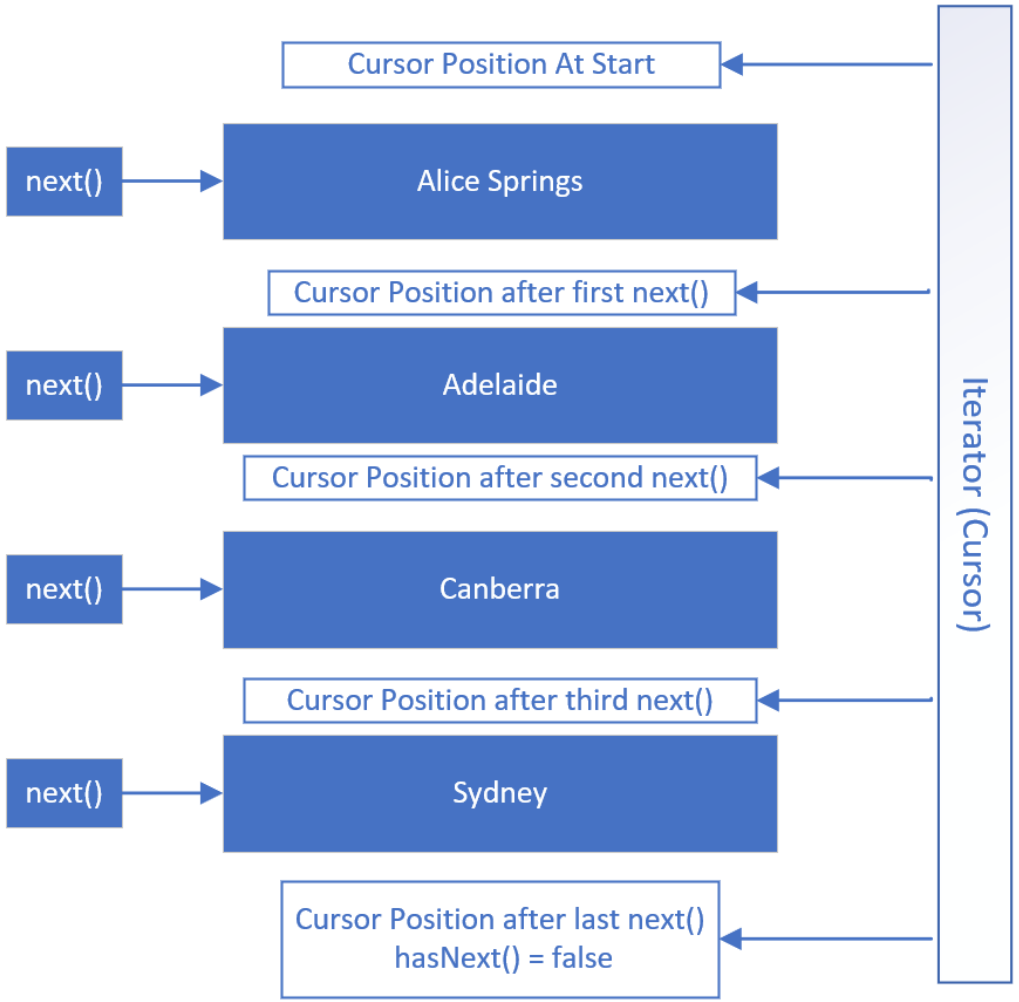


Iterator

- If you're familiar with databases, you might be familiar with a database cursor, which is a mechanism that enables traversal over records in a database.
- An iterator can be thought of as something similar to a database cursor.
- Specifically, an iterator is an object that allows traversal over records in a collection.
- How it works:
  - When you get an instance of an iterator, you can call the next method to get the next element in the list.
  - You can use the hasNext method to check if any elements remain to be processed.
  - In the code, you can see a while loop which uses the iterator's hasNext method to determine if it should continue looping
  - In the loop, the next method is called and its value assigned to a local variable and the local variable printed out.
  - This prints each element in a list but does it through or via the iterator object.

Iterator

- This slide shows visually how an Iterator works using the PlacesToVisit List.
- When an iterator is first created, its cursor position is pointed at a position before the first element.
- The first call to the next method retrieves the first element and moves the cursor position to be between the first and second elements.
- Subsequent calls to the next method moves the iterator's position through the list, as shown, until there are no elements left, meaning hasNext = false.
- At this point, the iterator or cursor position is below the last element.

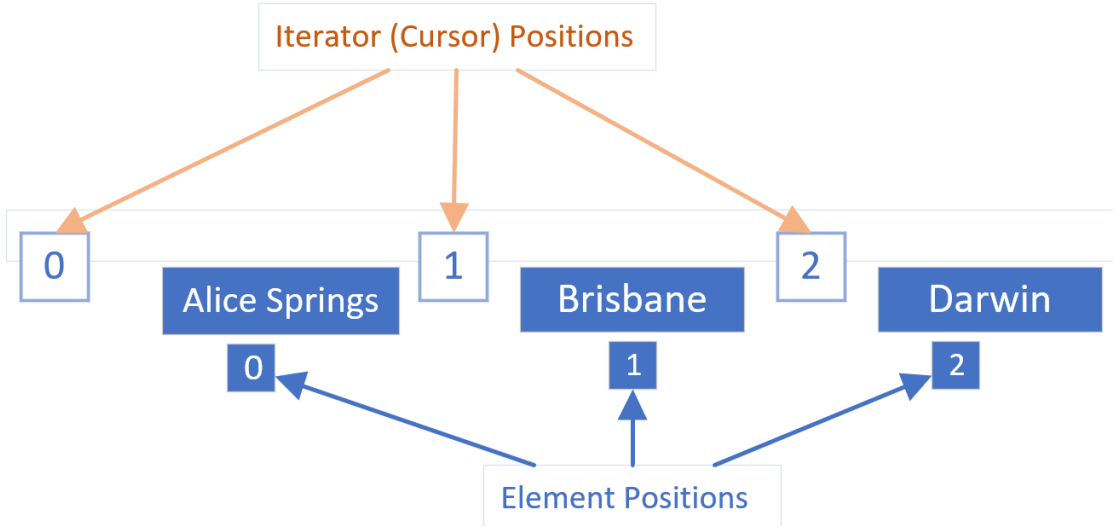


Iterator vs. ListIterator

- An Iterator is forwards only and only supports the remove method.
- A ListIterator allows you to navigate both forwards and backwards. Besides the remove method, it also supports the add and set methods, which function as you probably expect.

Iterator positions vs. Element positions

- It's really important to understand that the iterator's cursor positions are between the elements.




```
var iterator = list.listIterator();

String first = iterator.next();  // Alice Springs returned, cursor moved to
// cursor position 1
String second = iterator.next(); // Brisbane returned, cursor moved to cursor
// position 2

// Reversing Directions
String reversed = iterator.previous(); // Brisbane returned, cursor moved to
// cursor position 1
```

LinkedList Challenge

- I'm going to ask you to use LinkedList functionality to create a list of places, ordered by distance from the starting point.
- I want you to use a ListIterator to move both backwards and forwards through this ordered itinerary of places.
- First, create a type that has a field for a town or place name and a field for storing the distance from the start.
- Next, create an itinerary of places or towns to visit

Town	Distance from Sydney (in km)
Adelaide	1374
Alice Springs	2771
Brisbane	917
Darwin	3972
Melbourne	877
Perth	3923

LinkedList Challenge

- But this time, instead of Strings, you'll want to create a LinkedList of your place or town type.
- Here, I show a list of a few places in Australia and their distances from Sydney.
- You'll create a LinkedList ordered by the distance from the starting point, in this case, Sydney.
- Sydney should be the first element in your list.
- You don't want to allow duplicate places to be in your list for this data set.

Town	Distance from Sydney (in km)
Adelaide	1374
Alice Springs	2771
Brisbane	917
Darwin	3972
Melbourne	877
Perth	3923

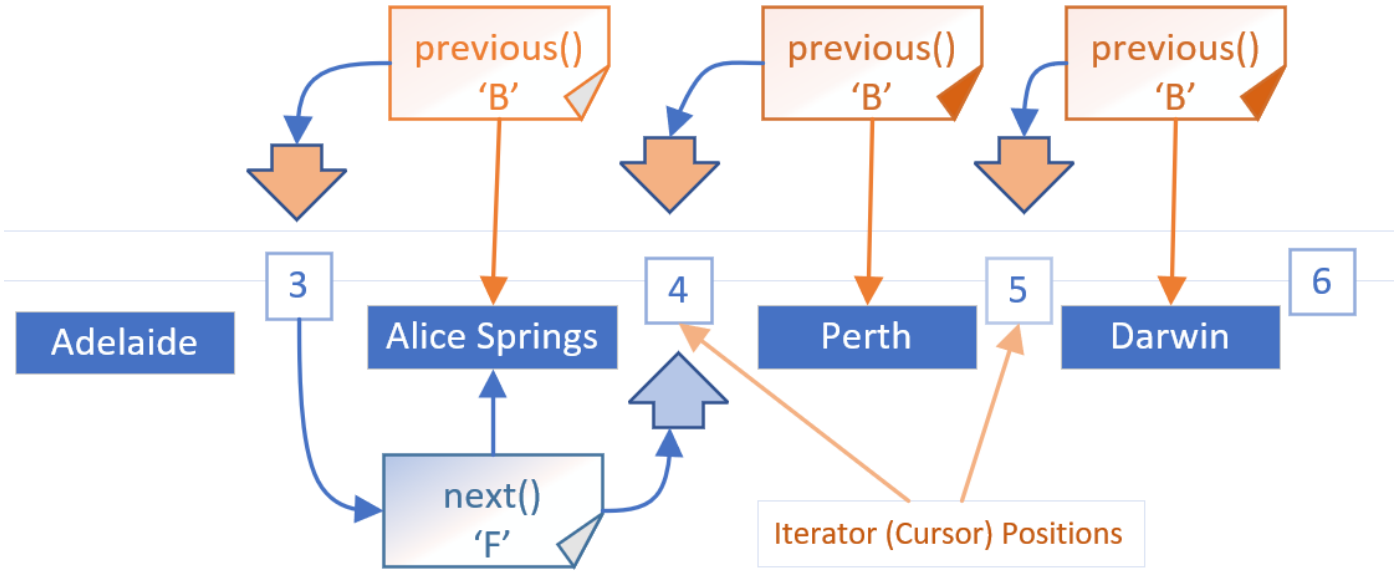
LinkedList Challenge

- In addition, you'll create an interactive program with the following menu item options.
- You'll want to use a Scanner and the nextLine method to get input from the console.
- You'll use a ListIterator to move forwards and backwards through the list of places on your itinerary.

```
Available actions (select word or letter):
(F)orward
(B)ackward
(L)ist Places
(M)enu
(Q)uit
```

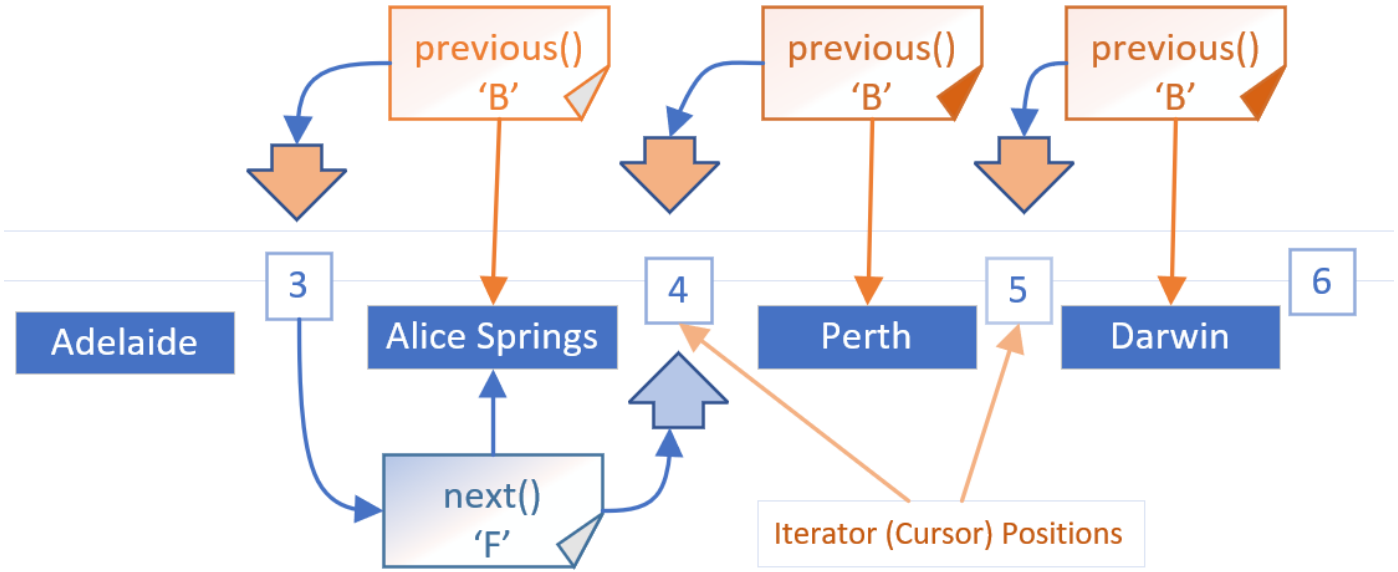
LinkedList Challenge

- Here, I show the three B or back menu items, each calling the previous method.
- This leaves our cursor position, shown as an orange arrow, positioned between Adelaide and Alice Springs.
- When we decided to go forward, we called the next method from this cursor position and that gave us Alice Springs.



LinkedList Challenge

- When we change directions, we have to compensate for the cursor being ahead of where we want it to be.
- So, we have to make the call, either next or previous, twice. The first time to adjust the cursor, and the second time to get the element.



Questions ?

